

TECHNICAL DEEP DIVE

突破长序列推理瓶颈：vLLM 分布式上下文并行架构解析

基于技术演讲与配套材料整理

配套材料：vLLM_DCP_PCP_技术分享.pptx

2026-08-10

原视频：[PCP 与 DCP 详解](#) · 配套资料：[CP-Viz 资料目录](#)

当大模型的输入上下文从 4K 增长到 128K 乃至百万量级，KVCache 的显存占用与 Attention 的计算量同步爆炸，传统的张量并行和数据并行均无法独立应对。vLLM 为此引入了 Context Parallelism（上下文并行）体系 - 在 Decode 阶段通过 DCP 将 KVCache 跨卡分散存储，在 Prefill 阶段通过 PCP 将输入序列切分并行计算，并配合 TPA 与动态路由机制将收益区间从超长序列下探至中短序列。本文沿着"为什么要切分 → 怎么管内存 → 数据如何流动 → 如何均衡负载 → 通信如何优化 → 实际能跑多快"这条因果链，逐层拆解其工程设计。

适读人群：熟悉大模型推理基础（如张量并行、KVCache），希望深入理解超长上下文分布式系统设计与性能调优的 AI 系统工程师。

前置知识：

- KVCache 机制与显存占用估算
- 张量并行（Tensor Parallelism, TP）基础
- GQA（Grouped-Query Attention）与 MLA（Multi-Head Latent Attention）架构差异
- AllGather、ReduceScatter 等基础集合通信原语

阅读目标：

- 理解 DCP 如何通过交错存储消除 KVCache 冗余
- 掌握 PCP 在 Prefill 阶段的序列切分与负载均衡策略
- 对比 GQA 与 MLA 架构在分布式上下文并行中的数据流差异
- 明确长序列推理中通信与计算的 Trade-off 及动态优化方案

1 长序列推理的工程矛盾与 CP 架构全景

本节要回答的问题

在超长上下文场景下，传统的 TP 和 DP（Data Parallelism，数据并行）为何失效？vLLM 如何从系统层面破局？

两道瓶颈同时收紧

先看两个核心推理指标：

指标	全称	受什么因素支配
TTFT	Time To First Token，首 token 生成时间	Prefill 阶段的计算量与并行度
TPOT	Time Per Output Token，逐 token 生成时间	Decode 阶段的 KVCache 访存与显存占用

序列变长带来两重压力：

1 **Prefill 阶段：Attention**

的计算量随序列长度呈二次增长，单卡算力不足以在可接受时间内完成全量预填充，TTFT 迅速劣化。

2 **Decode 阶段：每条请求的 KVCache (Key-Value**

Cache，注意力机制中缓存的历史键值对) 体积与序列长度成正比。当上下文达到 128K 量级，一条请求的 KVCache 就可能占满一张卡的可用显存，批处理容量锐减、吞吐塌方。

TP 能切分模型权重，却无法减少单条请求在每张卡上的 KVCache 副本 - 各卡仍然各自保存完整的 KV 缓存，显存冗余未被消除。DP

将不同请求分给不同卡，无法解决单条长序列的显存与计算瓶颈。单纯叠加 TP 或 DP 均不能根治这一核心矛盾。

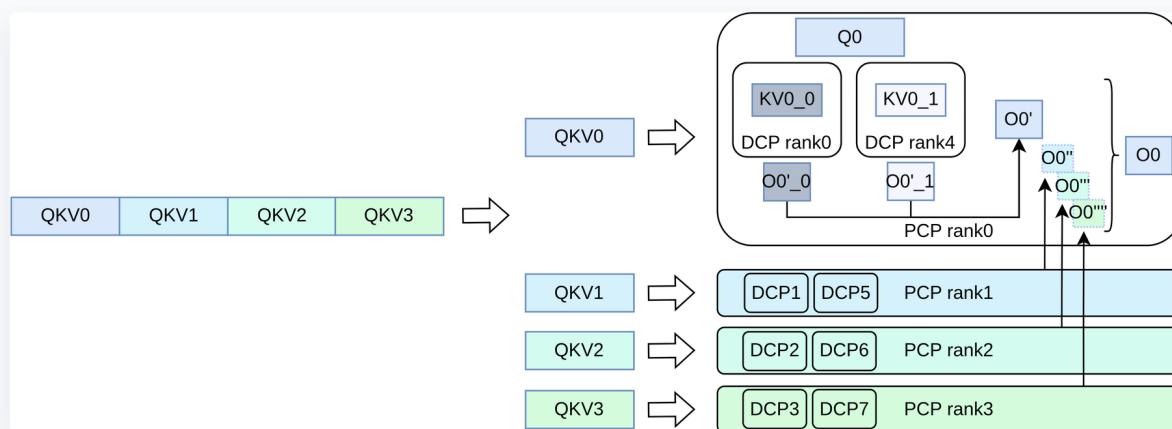
CP 架构全景：从序列维度切分

vLLM 的思路是沿**序列维度**做切分，将上下文并行拆为两个互补机制：

- **DCP (Distributed Context Parallelism, 分布式上下文并行)**：在 Decode 阶段将 KVCache 沿 token 维度分散存储到不同卡上，消除冗余副本，提升可支撑的最大序列长度和整体吞吐。该特性最初由月之暗面 (Moonshot AI) 的 ChaoHong 贡献 (PR #23734)。
- **PCP (Pipeline Context Parallelism, 流水线上下文并行)**：在 Prefill 阶段将输入序列切分为多个 chunk，分配到不同 PCP rank 并行计算，直接压缩 TTFT。

下图展示了 PCP 与 DCP 分别作用于推理流程的哪个阶段，以及二者如何协同 - 它是建立全局数据流直觉的入口：

图注：PCP 对 prefill 的输入序列进行切分，降低 TTFT；DCP 对 KVCache 进行序列切分，消除冗余存储，提升最大长度与吞吐。



图注：CP 特性总览 - PCP 在 Prefill 阶段切分输入序列，DCP 在 Decode 阶段切分 KVCache。来源：演讲 PPT 第 3 页

以图中的配置为例，逐步拆解各环节：

- 1 **输入序列切分**：原始输入的 QKV 被分为多段，分别送入不同 PCP rank。每个 rank 只计算全序列的一个子集，TTFT 近似按 rank 数倍降低。
- 2 **KVCache 跨卡分布**：每个 DCP rank 只持有部分 token 的 KV，不再保留完整副本。
- 3 **部分结果聚合**：Q 分别与不同卡上的 KV 分片计算得到中间输出，最终汇聚为完整结果。

因果链

序列变长 → KVCache 膨胀（显存墙）+ Attention 计算激增（算力墙）

→ DCP 沿 token 维度分散 KVCache → 单卡显存占用降至 1/N

→ PCP 切分输入序列 → Prefill 计算并行化 → TTFT 降低

→ 两个机制正交组合 → 显存与计算的双重解绑

值得注意的是，DCP 对 Prefill 阶段的推理计算本身没有影响 - 它改变的只是 KV 写入 KVCache 时的存储映射方式（slot mapping 与 block table 的修改），不会给 Prefill 增加额外计算开销。

最小直觉示例

假设一条 8K token 的请求，系统有 4 张卡构成一个 DCP 组：

- **无 DCP**：每张卡各存一份完整的 8K KVCache → 总冗余 4 倍。
- **启用 DCP**：每张卡只存 2K token 的 KVCache → 单卡显存降至原来的 1/4，释放的空间可容纳更多并发请求。

结论：CP 架构的本质是打破“每张卡保存完整 KVCache”的隐含假设，从序列维度实现存储与计算的分布式切分。DCP 解决显存瓶颈，PCP 解决计算瓶颈，二者正交叠加。

2 DCP 核心设计：交错存储与虚拟块管理

本节要回答的问题

KVCache 按序列维度切分并分散到多张卡之后，原本单卡上连续的 Block Table 需要感知“哪些 token 属于我、哪些不属于我”。如果管理粒度选择不当，要么出现大量只填了几个 token 的空洞 Block，要么在 PD 分离（Prefill-Decode Disaggregation，将预填充与解码分离到不同节点的架构）场景下传输大量无效数据。DCP 通过两个关键设计来解决这一矛盾：**Virtual Block（虚拟块）** 与 **Interleave Size（交错粒度）**。

Virtual Block：跨卡块的逻辑合并

在标准 vLLM 中，每张卡维护一份镜像的 Block Table，内容完全一致。引入 DCP 后，不同卡存放不同 token 的 KV，Block Table 不再对称。为了让上层的块分配与 Prefix Caching 命中逻辑继续正确工作，DCP 引入了 Virtual Block - 将同一个 CP 组内各 rank 对应的物理块合并为一个逻辑虚拟块，虚拟块大小为 `block_size × CP_size`。

上层按请求的总序列长度分配虚拟块时，能够精确推算出本卡实际需要的物理块数，不会因跨卡分布导致分配偏差。同时 Prefix Caching 在匹配前缀时以虚拟块为单位比较，确保命中判定的完整性。

Interleave Size：控制交错粒度

Token 按什么粒度分配到各 rank? DCP 用 **Interleave Size** 参数控制。给定一个 token ID，它被分配到的 rank 由以下公式决定：

变量说明： token_id 为该 token 在序列中的位置编号；interleave_size 为每次连续分配给同一 rank 的 token 数量；CP_size 为 CP 组的卡数。

rank = $\lfloor \text{token_id} / (\text{interleave_size}) \rfloor \bmod \text{CP_size}$

数字验证 (interleave_size = 4, CP_size = 4)：

Token 范围	计算过程	目标 Rank
tok 0 - 3	$\lfloor 0/4 \rfloor \bmod 4 = 0$	rank 0
tok 4 - 7	$\lfloor 4/4 \rfloor \bmod 4 = 1$	rank 1
tok 8 - 11	$\lfloor 8/4 \rfloor \bmod 4 = 2$	rank 2
tok 12 - 15	$\lfloor 12/4 \rfloor \bmod 4 = 3$	rank 3
tok 16 - 19	$\lfloor 16/4 \rfloor \bmod 4 = 0$	rank 0 (新一轮)

每 4 个连续 token 被分配到同一张卡，四组一轮后回到 rank 0，形成交错循环。

图解：Block Table 与 Slot Mapping 的实际映射

下图是理解 DCP 写入逻辑的关键 - 它展示了交错粒度为 4 时，各 rank 的物理槽位如何分布，以及 Slot Mapping 索引如何区分"该存"与"该忽略"的 token：



图中上半部分展示了一个 Virtual Block 内 tok0 - tok63 的分布 - 每 4 个 token 被连续放置在同一个 rank 的物理槽位中，四个 rank 依次交替。下半部分的 Slot Mapping 表展示了具体的索引映射：

- **Decode Token D20** (token_id=20) : $\lfloor 20/4 \rfloor \% 4 = 1$ ，落在 **rank 1**，映射到物理 slot 384 - 387 区间。
- **Decode Token D27** (token_id=27) : $\lfloor 27/4 \rfloor \% 4 = 2$ ，落在 **rank 2**。
- **不属于本卡的 Token**：其 Slot Mapping 值被设为 -1，后端写入 KVCache 时遇到 -1 直接跳过，不做任何存储操作。

每张卡只需扫描一遍 Slot Mapping 数组即可区分"该存什么"和"该忽略什么"，无需额外的跨卡协调。

PD 分离场景的实际考量

DCP 最早的实现采用 `interleave_size = 1`，即逐个 token 轮询分配。这在 PD 分离场景中会产生显著的通信浪费：PD 分离架构以整个 Block 为单位在 Prefill 节点与 Decode 节点之间传输 KVCache。假设 `block_size = 16`、`CP_size = 4`，当只有 16 个 token 时，一个 Block 理论上就能装下。但若 `interleave_size = 1`，这 16 个 token 被分散到 4 张卡各 4 个 token，每张卡的 Block 只填充了 25%，却不得不全部传输 - 通信量放大 4 倍，产生 75% 空洞。

解决办法是将 `interleave_size` 设置为 `block_size`。前面的 Block 尽可能被填满后才轮转到下一张卡。只有最后一张卡可能出现未填满的 Block，空 Block 完全不需要传输，从而大幅降低通信开销。

上述 PD 分离场景的配置建议主要来自演讲者的口头阐述，PPT 中未详细展开该配置的定量效果。

小结与边界条件

Virtual Block 与 Interleave Size 共同构成 DCP 的静态内存映射层：前者让块分配和前缀匹配在逻辑上保持一致，后者在内存连续性与通信效率之间提供可调旋钮。需要注意：当 `interleave_size` 不能整除 `block_size` 时，Block 内部可能出现不同 rank 的 token 交叉 - 演讲材料未给出该场景的具体处理细节。

3 架构差异下的 DCP 数据流：GQA 与 MLA

本节要回答的问题

内存映射解决了"数据存放在哪"的问题。接下来的关键是：Decode 阶段，每张卡只持有序列的一部分 KVCache，当前卡的 Query 如何"看到"其余卡上的历史 KV 信息？GQA 与 MLA 给出了截然不同的答案 - 前者移动 Q，后者移动 KV。理解二者的分歧，是把握 DCP 通信开销的关键。

前提：DCP 的硬性参数约束

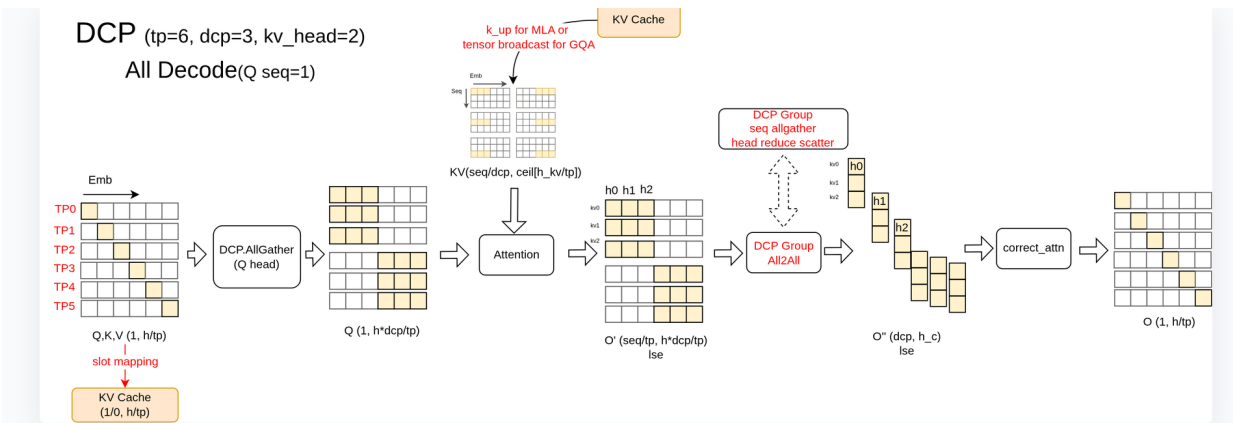
在讨论数据流之前，必须明确一条配置红线：

$DCP_size \times KV_head \leq q \ TP_size$

DCP 存在的出发点，正是当 TP size 大于 KV head 数量时，不同卡上会出现 KVCache 冗余存储。GQA 模型的 KV head 通常为 2、4 或 8，一旦 TP 规模超过该值，冗余不可避免。MLA 更为极端 - 经压缩后 KVCache 在所有卡上完全冗余（可等效视为 KV head = 1）。违反上述不等式的配置会使 KVCache 无法在 DCP 组内无重叠地切分，系统在初始化时即会拒绝。

GQA 路径：对 Q 做 AllGather

下图展示了 TP=6、DCP=3、KV_head=2 这一典型配置下的 Decode 数据流 - 它是理解 GQA 路径中张量形状如何逐步变换的核心示意图：



图注：GQA Decode 数据流（TP=6, DCP=3, KV_head=2, Q seq=1）。来源：演讲 PPT 第 12 页

图中核心路径分三步：

阶段	操作	张量形状变化	说明
1) Q 扩展	DCP.AllGather (Q head 维度)	$(1, h/tp) \rightarrow (1, h \times dcp/tp)$	组内各卡交换 Q head，使每卡 Q 覆盖同组所有 head
2) Attention	本地计算	$Q (1, h \times dcp/tp) \times KV (seq/dcp, h/tp) \rightarrow O'$	每卡用扩展后的 Q 与本地 KV 分片做注意力
3) 输出聚合	DCP Group All2All	$O' \rightarrow O (1, h/tp)$	seq 维度 AllGather + head 维度 ReduceScatter，经 correct_attn 修正

因果链：TP 将 Q head 切分到各卡 → DCP 又在序列维度切分 KVCache → 单卡的 Q 片段只能匹配本地 KV 的 head，看不到其他卡的序列分片 → 必须先在 DCP 组内 AllGather Q head，让每张卡持有足够的 Q 信息去读取本地 KV → Attention 后再通过 All2All（全互联通信操作，各卡同时向其余所有卡发送与接收数据）把分散的部分结果汇总为完整输出。

最小例子：6 张卡编号 0 - 5，DCP=3 将卡 {0,1,2} 归为一个 DCP 组。卡 0 原本只有 $h/6$ 个 Q head，AllGather 后获得 $h \times 3/6 = h/2$ 个 head。同时卡 0 只存约 1/3 的序列 KVCache。Attention 算出局部 O' 后，三张卡执行 All2All：序列维度做 AllGather 拼回完整序列，head 维度做 ReduceScatter 还原到 $h/6$ 。最终每卡得到正确的 $(1, h/tp)$ 输出。

MLA 路径：聚合 KVCache 而非移动 Q

MLA 的处理方式与 GQA 存在本质差异。讲者在演讲中明确指出，PPT 第 12 - 13 页关于"DCP 在 head 上做 AllGather"的图示对 MLA 情形存在绘制失误 - MLA 路径不对 Q 执行 AllGather。

实际流程如下：

- 1 **KVCache 分散存储**：与 GQA 相同，各卡按 interleave 方式只持有部分序列的 KV，形状为 (seq/dcp, h_c) (h_c 为 MLA 压缩后的隐维度)。
- 2 **reorg_kvcache**：通过 workspace 机制，先对 DCP 组内各卡的 KVCache 执行序列维度 AllGather，再执行 local_gather 操作，将交错存储的 KVCache 重排为按请求连续排列的格式，同时去除因不同卡存储长度不一致而产生的空 padding。
- 3 **本地 Attention**：重排后的 KVCache 紧凑连续，直接与本卡自身的 Q 做 Attention 计算，无需对 Q 做任何通信。
- 4 **迭代聚合**：通过 workspace 分批迭代（每次取一部分 KV），逐轮计算并累积，避免一次性加载过长上下文导致激活值显存爆炸。

两条路径的权衡对比

维度	GQA 路径	MLA 路径
通信对象	Q (head 维度)	KVCache (seq 维度)
通信量特征	Q seq=1 时数据量小	KVCache 随序列增长，通信量较大
额外计算	All2All + correct_attn	reorg_kvcache 重排 + 分批迭代
设计动机	Q head 多、KV head 少，移动 Q 更经济	MLA 压缩后 KV head ≈ 1 ，移动 Q 无意义；复用 workspace 分批模式更自然

核心取舍在于：GQA 的 KV head 少而 Q head 多，Decode 时 Q 序列长度仅为 1，AllGather Q 的通信量远小于搬运整段 KVCache；MLA 经压缩后 KV head 等效为 1，对 Q 做 AllGather 既无 head 可扩展，也不符合 MLA 已有的 workspace 分批模式，因此转而聚合 KV。

MLA 路径中 reorg_kvcache 的内部实现涉及跨 rank 的交错索引映射，讲者亦提及此处是代码中较难阅读的部分，具体细节需结合源码进一步核验。

4 PCP 核心设计：首尾拼接的序列切分

本节要回答的问题

DCP 解决了 Decode 阶段的显存与吞吐问题，但长序列的 Prefill 阶段仍面临全量 Attention 的算力瓶颈。PCP 沿序列维度切分输入来并行化 Prefill 计算 - 然而，Attention 携带 Causal Mask（因果掩码，即只允许当前 token 关注自身及其之前位置的下三角矩阵），朴素的连续切分会导致各卡计算量严重失衡。PCP 如何消除这一不均衡？

不均衡的根源

将长度为 N 的序列均分为 K 个连续块，第 i 块 (i 从 0 起) 承担的有效 Attention 计算量大致正比于它在下三角矩阵中覆盖的面积。第 0 块只有极窄的三角区域，第 $K-1$ 块几乎填满整个方形区域。把连续块直接分给不同 rank，最后一张卡的负载可能数倍于第一张卡，GPU 利用率因"等最慢者"而大幅下降。

首尾拼接策略

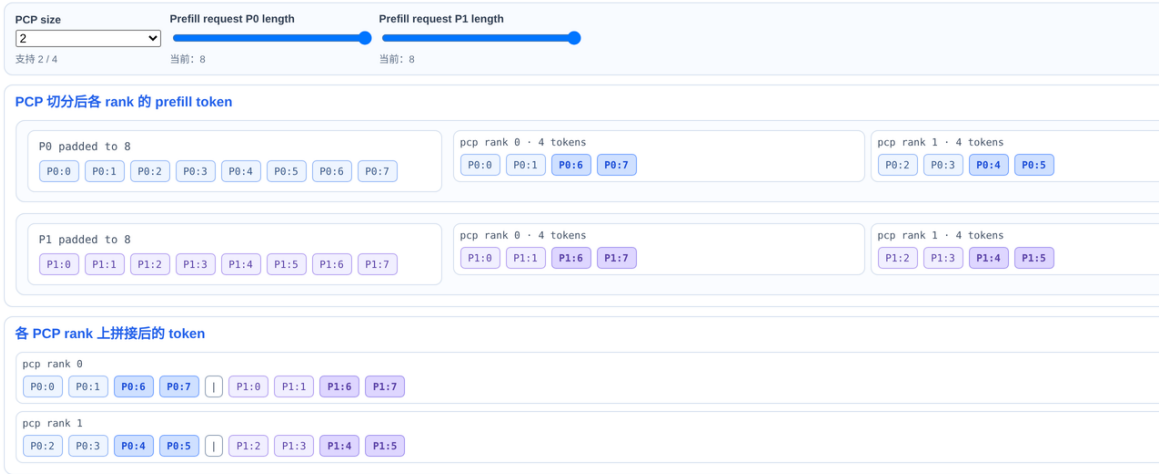
PCP 采用一种称为 **Chunk Swap** (也常被叫作 Zigzag) 的切分方式：先将序列等分为 $2K$ 个小块，然后将编号对称的首块与尾块配对，分配到同一个 rank。以 $PCP\ Size = 3$ 为例，序列被切成 6 个 Chunk，分配规则如下：

Rank	持有的 Chunk 编号	直觉
0	Chunk 0 + Chunk 5	最浅 + 最深
1	Chunk 1 + Chunk 4	次浅 + 次深
2	Chunk 2 + Chunk 3	中间两块

每对首尾块在 Causal Mask 下三角中所覆盖的有效计算面积近似互补：首块面积小、尾块面积大，二者之和趋于相等，使所有 rank 上的 FlashAttention（一种内存高效的 Attention 算法）计算量基本一致。

最小状态演进：PCP Size = 2

下图展示了两条 Prefill 请求在 $PCP\ Size = 2$ 时的完整切分过程 - 它是理解 Chunk Swap 在多请求场景下如何实际运作的关键：



图注：PCP 切分后各 Rank 的 Prefill Token 分布（PCP Size = 2）。来源：演讲 PPT 第 18 页

逐步拆解（以请求 P0 长度为 8 个 token 为例）：

- 1 **对齐**：每条请求的长度先被对齐到 $2 \times \text{PCP Size} = 4$ 的整数倍。此例中 P0 长度恰好为 8，无需额外填充。
- 2 **等分 Chunk**：P0 的 Token 0 - 7 被切成 4 个 Chunk，每 Chunk 含 2 个 token：Chunk 0 = [0, 1]，Chunk 1 = [2, 3]，Chunk 2 = [4, 5]，Chunk 3 = [6, 7]。
- 3 **首尾配对**：Rank 0 获得 Chunk 0 (Token 0, 1) 与 Chunk 3 (Token 6, 7)；Rank 1 获得 Chunk 1 (Token 2, 3) 与 Chunk 2 (Token 4, 5)。
- 4 **最终排布**：Rank 0 上拼接后的序列为 [0, 1, 6, 7]，Rank 1 上为 [2, 3, 4, 5]。Rank 0 的首段 (Token 0, 1) 在 Causal Mask 下只关注自身的极小三角区域，但尾段 (Token 6, 7) 需关注之前大量 token，二者计算量一小一大；Rank 1 的两段处于序列中段，计算量适中。两个 Rank 的总有效 Attention 面积因此近似相等。

切分引发的元数据变化

首尾拼接不是简单地"少发一半 token"，它会改变每张卡上几乎所有 Attention 相关的元数据：

- **Position IDs**：不再是单调递增序列，而是首段位置与尾段位置的拼接，直接影响 RoPE 等位置编码的计算。
- **Q Length**：每个 rank 上的 Query 长度大约折半。
- **QV Length (历史 KV 长度)**：需要做对应的截断与重组；首段对应的历史 KV 较短，尾段较长，分布不对称。

PCP 通信组拓扑

PCP 引入了独立于 TP 的通信组。在 rank 划分层级上，PCP 的切分先于 TP 执行：物理卡先按 PCP 维度分组获得不同的序列片段，再在组内按 TP 维度做 Attention Head 切分。下图帮助理解各通信组之间的嵌套与复用逻辑：



图注：Physical Rank Grid 及各通信组的形成规则。来源：演讲 PPT 第 16 页

图中左侧以行列网格表示物理 rank（行对应 PCP，列对应 TP），右侧表格列出了各 TP、PCP、DCP 通信组所含的 rank 编号。需要关注的是：当 DCP Size 等于 PCP Size 时，DCP 通信组与 PCP 通信组重合，KVCache 的序列切分仅在 PCP 组内执行。PCP 本身并不控制 KVCache 的交错存储，这一职责仍由 DCP 承担。

小结与边界

首尾拼接策略以极低的实现成本（仅需重排 token 索引与元数据）有效抹平了 Causal Mask 带来的计算倾斜，是 PCP 扩展到多卡 Prefill 的关键前提。但它带来一个直接后果：每个 rank 上的 Query 序列是不连续、不完整的。当输入序列超长、需要进一步做 Chunked Prefill（分块预填充，将过长的输入序列分为多个 chunk 逐步处理）时，不连续的 Q 会使分块边界和通信模式变得更加复杂。

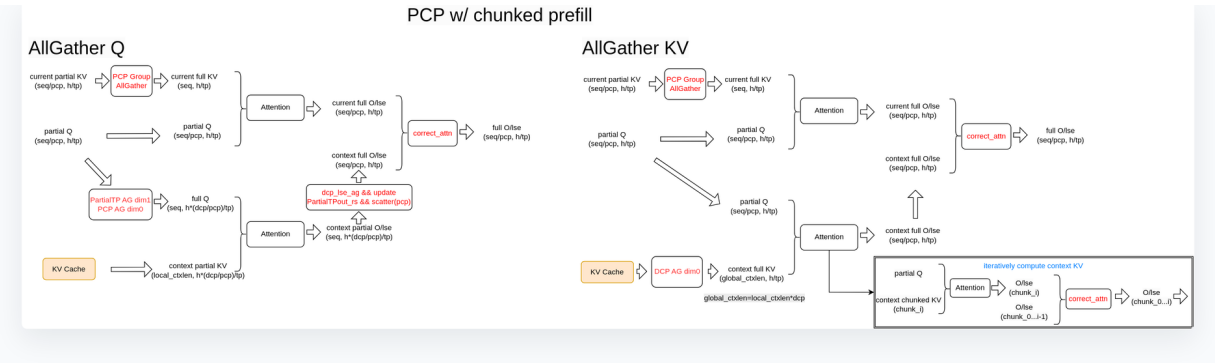
5 长 Prefill 的复杂通信：AllGather Q 与 KV 的抉择

本节要回答的问题

在 DCP 的 Decode 场景中，每张卡持有完整的 Q 序列 - 只是 KVCache 被切分。但引入 PCP 后，Q 的序列维度也被切分了。这意味着在 Chunked Prefill 场景下，单卡既没有完整的 Q，也没有完整的 KVCache，无法独立完成全局 Attention 计算。如何补全缺失的一侧？

两条路径的数据流对比

vLLM 为此实现了两条通信路径：AllGather Q 和 AllGather KV，分别通过补全不同侧的数据来还原完整的 Attention 结果。下图将两种方案并排呈现 - 左侧为 AllGather Q，右侧为 AllGather KV，是把握二者核心差异的关键：



图注：PCP 在 Chunked Prefill 下的两种通信方案 - 左侧为 AllGather Q，右侧为 AllGather KV。来源：演讲 PPT 第 23 页

维度	AllGather Q (左)	AllGather KV (右)
首次通信	PCP 组内对 Q 做 AllGather，得到 Full Q	DCP 组内对 KV Cache 做 AllGather，得到 Full KV
Attention 计算前提	Full Q + Partial KV → 退化为 DCP 长 Prefill 流程	Partial Q + Full KV → 退化为 TP Prefill 流程

维度	AllGather Q (左)	AllGather KV (右)
后续聚合	在 DCP 组中做 KV 侧通信，累加上下文信息	当前 Q 与全量 KV 直接计算，无需额外序列补全

AllGather Q 路径：补全查询序列

核心逻辑是先让每张卡看到完整的 Q，再沿用已有的 DCP 流程处理 KV 侧的分布式计算：

- 1 **PCP 组 AllGather**：各卡将自己持有的 Partial Q 在 PCP 组内拼接为 Full Q。
- 2 **Local Attention**：Full Q 与本卡持有的 Partial KVCache 做一次 Attention，得到局部结果。
- 3 **DCP 组通信**：在 DCP 组内将其他卡的 KVCache 信息汇聚过来，等价于获取了完整上下文。
- 4 **结果聚合**：将局部 Attention 输出与上下文 Attention 输出合并，得到最终结果。

AllGather KV 路径：迭代补全上下文

思路相反 - 先让每张卡看到完整的 KVCache，Q 虽然是局部的但无需补全：

- 1 **DCP 组 KV AllGather**：收集所有卡的 KVCache，使每卡获得全量上下文。
- 2 **Local Attention**：Partial Q 与 Full KV 做计算，Q 只查询自己对应的那段序列，计算结果直接就是最终结果。
- 3 **输出更新**：将当前 chunk 的 QKV 计算结果与上下文部分做一次更新。

图中右侧标注的"Iterative Computation"对应着长上下文场景中 KVCache 分块传输、逐步累加的过程。该方案复用了 MLA 中已有的 AllGather workspace 和 DCP 通信逻辑。

为什么需要两套方案？ - 通信量的翻转点

两条路径的通信开销取决于 Q 和 KV 各自的数据量，而这两个量在不同工况下此消彼长：

- **AllGather Q 的通信量**正比于 Q 的序列长度。在 Chunked Prefill 中，当 `max_num_batched_tokens` 设得较大时，当前 chunk 的 Q 会很长，通信代价上升。
- **AllGather KV 的通信量**正比于上下文的 KVCache 大小。但 KV 经过 head 维度切分后，在上下文较短时，实际数据量可能比一个完整 chunk 的 Q 还小。

翻转点出现在：当 batch 较大、chunk 较长时，Q 的数据量超过切分后的 KV，此时 AllGather KV 更优；反过来，当上下文极长而当前 chunk 较短时，AllGather Q 的通信量更小。

演讲材料未给出具体的通信量公式或量化对比数据，上述分析基于演讲者对两种方案适用场景的定性描述。

小结与适用边界

PCP 在 Chunked Prefill 场景下的通信问题，本质是"Q 不完整"和"KV 不完整"叠加后的双重困境。AllGather Q 将问题归约为 DCP 流程，AllGather KV 将问题归约为 TP 流程，两者互补而非替代，理想的工程实现应根据运行时的 batch token 数与上下文长度动态选择。

需要注意：学术界常用的 Ring Attention（一种通过环形通信传递 KVCache 的长序列 Attention 方法）理论上也可解决类似问题，但 vLLM 目前尚未采用该方案。AllGather Q 方案已合入主线，AllGather KV 方案（尤其针对 MLA 架构）正随后续 PR 推进中。

6 突破短序列劣化：TPA 优化与动态路由

本节要回答的问题

至此，长序列的 Prefill 和 Decode 均有了对应的并行策略。然而 CP 特性在 Decode 阶段至少引入两次额外的集合通信。当序列较短时，这些通信的耗时远超 KVCache 切分带来的计算收益，导致 TPOT 反而恶化。实测表明，**序列长度需超过 128K 才能观测到 TPOT 的正向收益**。如何把 CP 的有效区间从超长序列下探到中短序列？

劣化根因：隐藏维度上的 AllGather 代价

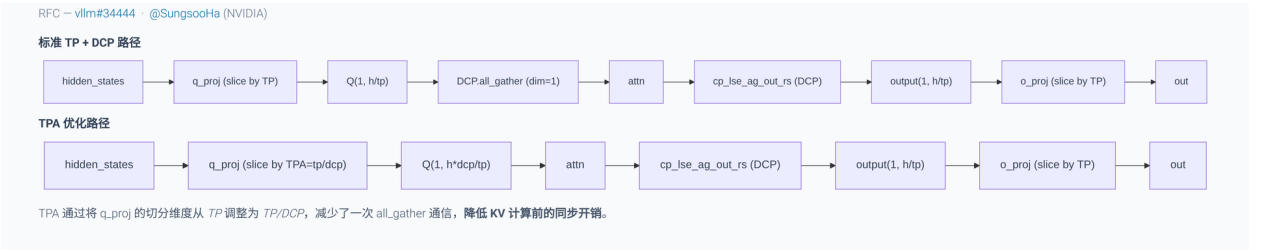
在标准 TP+DCP 路径中，`q_proj`（Query 投影层）的权重按 TP 维度切分。计算完成后，需要在隐藏维度（hidden dimension）执行一次 AllGather，将分片结果拼回完整张量，才能进入后续 KV 计算。

这次 AllGather 的代价不仅是通信本身。由于拼接发生在隐藏维度而非最外层维度，运行时往往需要先执行一次 transpose 或 contiguous 操作使内存布局连续，由此引入多个额外算子。Decode 阶段本身是访存瓶颈（memory-bound），这些额外算子对 TPOT 的影响十分显著。

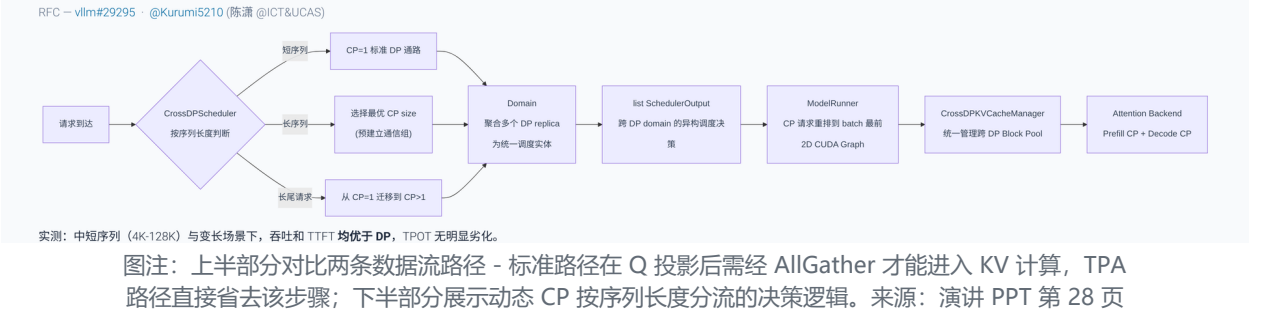
TPA：用冗余计算换取通信消除

TPA（Tensor Parallel Size Attention）的核心思路是：在 `q_proj` 层将切分粒度从 TP 调整为 TP/DCP。这样每张卡在完成 Q 投影后直接获得完整的 TP size 输出，无需再做隐藏维度上的 AllGather 及配套的 transpose 操作。

下图的上半部分对比了标准路径与 TPA 路径的差异，下半部分展示了动态 CP 的调度逻辑 - 二者分别从算子层面和调度层面解决短序列劣化问题：



动态 CP 特性



阶段	标准 TP+DCP 路径	TPA 优化路径
q_proj 切分维度	TP	TP / DCP
Q 投影后通信	AllGather + transpose	无
进入 KV 计算前状态	完整 hidden_states	完整 hidden_states
额外计算量	无	Q 投影计算量增大（权重未充分切分）

代价是 Q 投影层的计算量增加，因为每张卡需处理更大的权重分片。但 Decode 阶段属于访存瓶颈，这部分额外计算可被流水线掩盖，对整体时延影响可控。最终效果：**减少一次 AllGather 及其附带算子开销，使 TPOT 劣化显著收窄。**

TPA 特性在演讲时已提交 RFC 和 PR，但尚未合入 vLLM 主线，具体合入版本待核验。

动态 CP：按序列长度自适应路由

TPA 降低了单次通信的成本，但对于真正的短序列（如 4K 以下），任何 CP 通信都是净损耗。动态 CP 从调度层面解决这一矛盾：**让系统在 DP 与不同 CP size 之间按需切换**，而非全局锁定一个并行策略。

参照上图下半部分的流程，请求到达后经历三条通路：

- 1 **短序列** → 走 CP=1 的标准 DP 通路，不引入任何 CP 通信，避免劣化。
- 2 **长序列** → 根据预设阈值选择最优 CP size，将多个 DP replica 聚合为一个 CP 组，获取 KVCache 切分收益。
- 3 **长尾请求**（持续执行 Decode 或长 Prefill 的存量请求） → 支持从 CP=1 动态迁移至 CP>1 的通路，在运行时完成通信策略切换。

效果与边界

经过 TPA 与动态 CP 的组合优化，CP 特性的优势序列区间从 32K 下降至 4K。在 4K - 128K 的中短序列以及变长混合场景下，系统能在吞吐与时延之间取得更好的平衡。

需要注意：

- 演讲材料未给出 TPA 优化后的精确 TPOT 对比数值，上述结论来自演讲者的口头总结。
- 动态 CP 的阈值配置（何时从 DP 切换到 CP、选择哪个 CP size）依赖预设参数，自动调优策略演讲中未展开。
- 动态 CP 特性在演讲时仍处于 PR 阶段，尚未完全合入主线。

一句话结论：TPA 在算子层面削减通信次数，动态 CP 在调度层面隔离短序列的通信损耗，二者协同将从"仅限超长序列的特殊手段"拓展为"中短序列也能受益的通用能力"。

7 性能边界与部署实践指南

本节核心问题

在真实生产环境中，DCP 与 PCP 分别能带来多大收益？它们各自的适用边界在哪里？

DCP：解锁超长序列的关键开关

DCP 最直观的收益体现在单序列可推理长度的突破。由于 KVCache 被切分存储到多张卡上，单卡显存不再构成上下文长度的硬性天花板。

在演讲中提及的实测案例里，满足以下硬件条件时，开启 DCP 可将单序列推理长度推至 1M token：

硬件环境	卡数	可达序列长度	备注
华为 910C（单机 A3）	8 卡	1M token	演讲 PPT 明确提及
NVIDIA H200	8 卡	1M token	演讲者引用社区测试结果

限定条件：上述 1M token 为单序列场景下的上限，基于演讲材料中提及的"常见模型"（未给出具体模型名称与参数量）。当同时服务多条较短序列时，DCP 的核心价值转变为提升 KVCache 总容量，从而承载更高并发。

PCP：DP 与 TP 之间的均衡策略

PCP 的性能定位可以用一句话概括：时延优于 DP、略逊于 TP；吞吐优于 TP、略逊于 DP。它在两种经典并行策略之间开辟了折中路径。

这一定位的因果链如下：

- 1 **相比 DP**：PCP 将同一请求的序列拆分到多卡计算，每卡处理的有效 KV 量减少，单请求时延降低。但由于引入了额外的跨卡通信（如 AllGather），时延无法线性缩放到 DP 的理论下限，吞吐上也无法超越同等规模的 DP 部署。
- 2 **相比 TP**：TP 在每一层的矩阵运算中都需要同步通信，通信频次高；PCP 的通信仅发生在 Attention 阶段的 KVCache 层面，通信总量相对可控，吞吐表现更好。但 TP 每次计算天然负载均衡，PCP 受制于序列在不同 rank 间的分布差异，时延上仍有差距。

最佳收益区间：中长序列场景，输入长度大致落在 **32K ~ 256K token** 范围，且并发量接近系统最优吞吐时效果最显著。

不适用的场景与已知局限

- **短序列场景**：DCP 引入的额外通信会导致时延劣化，演讲 PPT 以 128K 作为时延收益由负转正的大致分界线（经 TPA 优化后可降至 4K）。
- **并发远超系统吞吐时**：优先扩 DP 实例数才是正解。PCP 更适合“并发量接近但未显著超过系统最优吞吐”的窗口期。
- **部分高级特性适配未完成**：演讲者明确提到，MTP / EAGLE（一种投机解码算法）等框架的 CP 适配仍处于早期阶段，DSV4 适配等特性的 PR 正在推进中。

架构选型决策路径

- 1 **序列长度是否超出单卡显存承载？** 是 → 必须开启 DCP。
- 2 **开启 DCP 后，是否需要进一步压低 TTFT？** 是，且并发压力适中 → 叠加 PCP。
- 3 **序列普遍较短（< 32K）且并发极高？** → 优先使用纯 DP，暂不引入上下文并行。

PCP 的核心价值，在于为那些“TP 吞吐不够、DP 时延太高”的中间地带提供一个可落地的工程选项。

结论与局限

核心结论

- 1 **DCP 和 PCP 正交组合，重构了长序列推理的内存与计算分布逻辑**。DCP 沿 token 维度分散 KVCache 消除显存冗余，PCP 以首尾拼接方式切分输入序列保证负载均衡，二者协同实现显存与计算的双重解绑。
- 2 **交错存储与虚拟块管理是 DCP 精细化切分的基础**。Interleave Size 在内存连续性与通信效率之间提供可调旋钮，在 PD 分离场景下对齐 Block Size 可大幅减少无效传输。
- 3 **GQA 与 MLA 架构需要差异化的 DCP 数据流**。GQA 通过 AllGather Q head 来读取分散的 KV；MLA 通过 [reorg_kvcache](#) 聚合分散的 KVCache 后做本地计算。核心取舍取决于 Q head 与 KV head 数量的相对关系。
- 4 **PCP 的 Chunk Swap 策略以极低的实现成本抹平了 Causal Mask 带来的计算倾斜**，使多卡 Prefill 的 GPU 利用率不因下三角矩阵的不对称性而大幅下降。
- 5 **Chunked Prefill 场景下的 AllGather Q 与 AllGather KV 两条路径互为镜像**，最优选择在不同 batch 大小与上下文长度下会发生翻转，工程实现应支持运行时动态切换。

- 6 **TPA 在算子层面削减通信次数，动态 CP 在调度层面隔离短序列损耗**，二者协同将 CP 的优势区间从 32K 下探至 4K，使分布式上下文并行从“超长序列专属”演进为更通用的能力。
- 7 **分布式推理正在从静态的并行策略走向基于请求特征的细粒度自适应路由架构。**
序列长度、并发量、模型架构共同决定了最优的并行策略组合，不存在一套参数适用所有场景的万能配置。

已知局限

- 本文的性能结论（如 1M token 上限、128K TPOT 收益分界线、4K 优势下限）均基于演讲者口头描述的特定硬件条件（单机 8 卡 H200 / 华为 910C），缺少系统化的量化对比图表，实际部署效果需结合具体模型和硬件验证。
- MLA 路径的 `reorg_kvcache` 实现细节、AllGather KV 方案的完整代码、TPA 特性以及动态 CP 的调度策略在演讲时（2026 年 5 月底）仍处于 PR 或 RFC 阶段，未完全合入 vLLM 主线，本文描述的机制与最终合入版本可能存在差异。
- MTP / EAGLE 等投机解码框架的 CP 适配、DSV4 适配等特性仍在推进中，使用时需关注社区最新进展。
- 演讲 PPT 第 12 - 13 页关于 MLA DCP 数据流的图示存在绘制失误（以讲者口头修正为准），读者如参考原始 PPT 需注意图文矛盾。